

LLM-Guided Performance Engineering for GPU Kernels

AIRI Summer 2026

1 августа 2026 г.

1 Постановка задачи

Проект исследует два аспекта построения LLM-агента, который разрабатывает и оптимизирует GPU-ядра внутри фреймворка, плохо знакомого модели.

Первое направление — освоение нового фреймворка прямо в процессе работы агента. Модели неплохо справляются с Triton, но испытывают трудности с менее распространёнными абстракциями, такими как CuTe. Чтобы понять, чем компенсировать этот пробел, нужно окружение, подающее модели документацию, примеры кода, результаты компиляции, тесты и данные профилирования; на нём уже можно сравнивать способы подачи информации и виды обратной связи. Ожидаемый результат — анализ инструментов, позволяющих LLM освоить новый GPU-фреймворк *без дообучения*.

Второе направление — незнакомые числовые форматы на примере FP8: переход с FP16 требует учёта специфики кодирования и специализированных инструкций, поэтому нужна отдельная обвязка для генерации FP8-кода и устойчивый замкнутый цикл разработки. Оба направления мы измеряем одними и теми же метриками: корректность, ускорение, число итераций агента и стоимость в токенах.

2 Инфраструктура эксперимента

Модель. Мы задеплоили Qwen3.6-35B-A3B в квантизации Q8_0 на сервере с 2×V100 с пайплайн-параллелизмом (PP = 2) и открыли доступ через OpenAI-совместимый эндпоинт. Впоследствии, мы перешли на внешнего провайдера - DeepSeek v4 Flash, так как Qwen совсем не умел справляться с задачей.

Оценщик. Кандидатное ядро — единственный файл, который агент имеет право править. Всё остальное принадлежит оценщику: генерация входных данных, эталонная реализация на Torch, проверка допусков, замер времени и вердикт PASS. Агент не может подменить проверку и не видит эталонных таймингов. Ядра исполняются удалённо на NVIDIA B300; для стабильности замеров делается несколько разминочных прогонов, затем несколько основных с усреднением. Перед отправкой на GPU кандидат проходит локальную статическую проверку `cute-check` (синтаксис и типы, без обращения к устройству) — она дёшево отсеивает заведомо нерабочий код.

Задачи. Готовых бенчмарков по CuTe DSL и FP8 не существует, поэтому мы адаптировали задачи из KernelBench: переиспользовали формат, но написали собственные эталонные решения, привели входы и эпилоги к FP8-семантике и зафиксировали свои пороги численного расхождения. Всего девять задач — первого уровня (одиночная операция) и второго (слитая последовательность: GEMM вместе с эпилогом). Эксперименты E1–E4 прогонялись на одной задаче второго уровня.

Справочные материалы. Документацию по CuTe DSL мы разбили на четыре кумулятивных уровня, не зависящих от конкретной задачи: *только задача* (постановка и стартовый файл), *+ документация* (CuTe DSL API, алгебра layout’ов, архитектура Blackwell, TMA и TMEM, численные свойства FP8/FP4), *+ примеры* (нейтральные фрагменты рабочего кода) и *+ разбор ошибок* (типовые классы ошибок и то, как они выглядят в диагностике). Каждый уровень включает предыдущий целиком, поэтому разница между соседними строками таблиц — ровно вклад добавленного материала.

3 Как устроены эксперименты E1–E4

Всего мы прогнали семь конфигураций. Первые четыре — лестница уровней справки, они отвечают на вопрос, *какое знание вообще работает* (E1). Оставшиеся три меняют ровно один параметр относительно лучшего уровня лестницы: способ доставки того же материала (E2), наличие второго агента-критика между ходами (E3) и возврат сводки профилировщика следующему ходу (E4). Конфигурация с разбором ошибок из E1 служит для них общей опорной точкой, поэтому каждое сравнение однопараметрическое.

Каждая конфигурация — один независимый запуск из шести *ходов*. Ход — итерация работы агента: он пишет или правит кандидатное ядро, прогоняет его через статическую проверку и через GPU, получает диагностику и переходит к следующему ходу. Эталонная реализация перемеряется внутри каждой конфигурации (0.3815–0.3835 мс), чтобы дрейф загрузки машины не искажал сравнение ускорений.

Как читать таблицы. «PASS $k/6$ » — в скольких из шести ходов получилось корректное ядро. «Лучшее» — время самого быстрого корректного ядра за запуск, «ускорение» — во сколько раз оно быстрее эталона. Вход разбит на общий и попавший в кэш. Каждая конфигурация прогонялась один раз.

4 E1 — какое знание важно

Лестница из четырёх уровней: от голой постановки задачи (195 токенов справки) до полного стека с разбором типовых ошибок (33 146 токенов). Вопрос — где проходит граница между «модель не может» и «модель может».

Таблица 1: E1, лестница уровней справки. Шесть ходов на уровень, время в мс.

Уровень справки	Справка, токенов	PASS	Ходов до 1-го PASS	Лучшее, мс	Ускор.	Вход	из них из кэша	Выход	Токены рассужд.	Токенов до 1-го PASS	Проверок на GPU
Только задача	195	0/6	—	—	—	10,323,585	9,936,512	61,096	198,552	—	8
+ документация	17,860	0/6	—	—	—	14,519,559	13,687,552	92,626	470,913	—	15
+ примеры	26,416	5/6	1	0.1181	3.23×	8,762,557	8,308,864	72,828	231,642	2,805,178	25
+ разбор ошибок	33,146	6/6	1	0.1160	3.29×	7,434,221	6,977,792	60,662	179,156	1,604,327	14

Граница проходит между документацией и примерами, и это не тот ответ, который подсказывает объём текста. Почти восемнадцать тысяч токенов связного описания API, алгебры layout’ов и архитектуры Blackwell не сдвигают результат ни на один ход: 0/6, ровно как на голой постановке. При этом конфигурация с документацией — *самая дорогая* из семи: 14.5 млн входных токенов и 470 913 токенов рассуждений, вдвое с лишним больше, чем на голой задаче. Модель добросовестно читает, думает дольше всех, пишет больше всех — и не решает ничего. Вредна она вполне определённым образом: ни одна из 21 проверки не прошла, и восемь отказов — это обращения к функциям, которых в библиотеке нет. Прочитав описание API прозой, модель приобретает словарь, но не приобретает синтаксиса. Без справки поведение другое: там кандидат до устройства просто не доходил. Без документации модель сдаётся, с документацией — галлюцинирует.

Перелом даёт добавление примеров: 5/6 и первое корректное ядро уже на первом ходу, ускорение 3.23×. Закрепляет его разбор ошибок — 6/6, все 20 проверок пройдены, ни одного отказа за запуск, лучшее ядро 0.1160 мс (3.29×). И лучший уровень оказался *самым дешёвым*: при максимальном объёме справки он потратил 7.43 млн входных токенов вместо 10.32 млн и 179 156 токенов рассуждений — меньше, чем любая другая конфигурация; до первого решения дошёл за 1.60 млн токенов против 2.81 млн у уровня с примерами. Вывод E1: полезность справки не пропорциональна её объёму и даже не растёт вместе с ним. Работают конкретные артефакты — рабочие фрагменты кода и разбор типовых ошибок; описательные главы про API не работают вовсе, но исправно тратят бюджет. Дальше уровень с разбором ошибок используется как опорная конфигурация.

5 E2 — как доставлять справку

E2 сравнивает два способа доставки *одного и того же* материала. В опорной конфигурации (**files**) файлы справки лежат в рабочей директории, и агент открывает их сам, когда сочтёт нужным. В альтернативе (**prompt**) весь материал целиком вставляется в промпт на каждом ходу. Содержимое идентично, различается только то, кто решает, что читать.

Таблица 2: E2, доставка справки: файлами в рабочей директории против вставки в промпт.

Конфигурация	Доставка	PASS	Ходов до 1-го PASS	Лучшее, мс	Ускор.	Вход	Вход на одно решение	Выход	Токенов до 1-го PASS	Таймауты
Разбор ошибок (E1)	files	6/6	1	0.1160	3.29×	7,434,221	1,239,037	60,662	1,604,327	0
E2	prompt	4/6	3	0.1080	3.54×	12,986,542	3,246,636	102,068	5,648,113	3

Результат расщепляется по метрикам. По пиковой производительности выигрывает вставка в промпт: 0.1080 мс и 3.54× — лучшее ядро за все семь конфигураций. По всему остальному она проигрывает:

устойчивость падает с 6/6 до 4/6, первое корректное ядро появляется только на третьем ходу, и это единственная конфигурация с таймаутами — три хода из шести исчерпали бюджет, не дойдя до проверки. Постоянное присутствие всего материала не даёт агенту «забыть» редкие конструкции, отсюда более высокий потолок оптимизации; но тот же материал вытесняет из внимания рабочий контекст — собственный код, диагностику компилятора, историю ходов. Косвенно это подтверждает характер отказов: обращений к несуществующему API нет ни одного (документация всегда перед глазами), зато есть пять ошибок компиляции — модель знает, *что* писать, но теряет нить того, *что она уже написала*.

Экономическая сторона однозначна: вход на одно полученное решение — 1.24 млн против 3.25 млн токенов, доставка файлами дешевле в 2.6 раза; до первого решения разрыв ещё больше, 1.60 млн против 5.65 млн. Практический вывод: чтение справки агентом по требованию — разумный режим по умолчанию, а вставку в промпт имеет смысл рассматривать точечно, на финальной стадии оптимизации уже работающего ядра, где пиковая скорость важнее устойчивости и цены.

6 E3 — обратная связь от агента-критика

E3 добавляет между ходами вызов второго агента — критика, работающего только на чтение. Он получает текущее кандидатное ядро, диагностику с V300 и тот же уровень справки, что и основной агент, и возвращает короткие подсказки. Сам он ничего не исполняет и не правит. Его токены учитываются отдельно — иначе сравнение стоимости было бы нечестным.

Таблица 3: E3, агент-критик между ходами. Токены критика учтены отдельно.

Конфигурация	PASS	Ходов до 1-го PASS	Лучшее, мс	Ускор.	Вход осн. агента	Выход осн. агента	Токены критика	Подсказок
Разбор ошибок (E1)	6/6	1	0.1160	3.29×	7,434,221	60,662	—	—
E3	4/6	1	0.1164	3.29×	12,519,376	96,038	100,722	13

Эффект нулевой по скорости и отрицательный по всему остальному. Лучшее ядро — 0.1164 против 0.1160 мс, то есть то же ускорение 3.29× с точностью до шума. Устойчивость падает с 6/6 до 4/6. Вход основного агента вырастает на 68 %, выход — на 58 %, и сверху добавляются 100 722 токена критика на 13 подсказок, около 7 750 токенов на подсказку. Мы платим примерно в полтора раза больше и получаем менее надёжный запуск.

Диагноз виден в характере отказов: из 11 неудачных проверок **десять — это обращения к несуществующему API**. Критик воспроизводит ровно ту патологию, которую мы видели в E1: он читает документацию, но ничего не исполняет, поэтому его подсказки правдоподобны по форме и называют функции, которых нет. Основной агент доверяет им больше, чем собственной обратной связи от компилятора: подсказка приходит как внешний авторитет, а сообщение об ошибке нужно ещё интерпретировать. Вывод: советчик без исполнения — это дополнительный источник галлюцинаций, а не фильтр от них. Если возвращаться к идее критика, у него должно быть право самому запустить *cute-check* или пробную компиляцию и высказываться только о проверенных гипотезах.

7 E4 — обратная связь от профилировщика

E4 возвращает следующему ходу компактную сводку по времени исполнения участков ядра, снятую на V300. Профилировать можно только работающее ядро, поэтому сводка появляется лишь *после* первого успеха: по построению она не может помочь пройти тест впервые, и ожидаемый эффект здесь — на ускорение среди уже проходящих ходов.

Таблица 4: E4, возврат сводки профилировщика следующему ходу.

Конфигурация	PASS	Лучшее, мс	Ускор.	Вход	Выход	Проверок на GPU
Разбор ошибок (E1)	6/6	0.1160	3.29×	7,434,221	60,662	14
E4	5/6	0.1133	3.38×	11,168,385	83,196	19

Так и вышло: доля успешных ходов практически не меняется (5/6 против 6/6), а лучшее ядро улучшается с 0.1160 до 0.1133 мс — с 3.29× до 3.38×, около 2.3 % по времени. Качество кода при этом ровнее:

за весь запуск всего два отказа, оба на компиляции, ни одного обращения к несуществующему API и ни одного численного расхождения.

Таблица 5: E4, лучшее время по ходам, мс. Конфигурация с профилировщиком долго отстаёт.

Ход	1	2	3	4	5	6
E4 (с профилировщиком)	0.2999	отказ	0.3050	0.1515	0.1269	0.1133
Разбор ошибок (E1, опора)	0.3711	0.1267	0.1261	0.1179	0.1164	0.1160

Информативнее итога — траектория. На первом ходу конфигурация с профилировщиком даже впереди, но затем проваливает второй ход и до пятого включительно остаётся позади; обгон происходит ровно на последнем, шестом ходу. Профиль — отложенная инвестиция: сначала нужно получить хоть какое-то работающее ядро, потом осмыслить его разбор, и только после этого начинается целенаправленная оптимизация вместо угадывания. Опорная конфигурация тем временем выходит на плато рано и почти не улучшается с третьего хода ($0.1261 \rightarrow 0.1160$).

Цена — 11.17 млн входных токенов против 7.43 млн (+50 %). На горизонте в шесть ходов это плохая сделка: два процента ускорения за пятьдесят процентов бюджета. Но форма кривых говорит, что мы видим только начало — опорная конфигурация свой потенциал исчерпала, а конфигурация с профилировщиком на последнем ходу всё ещё улучшается на 11 % за ход. Проверять это нужно увеличением числа ходов, а не повторением текущей конфигурации.